

Lab 7

Part A - Classes and objects

1. Create a **Book** class with **title**, **author** and **pages**. Create at least four Book objects and print their attributes.
2. Create a **Laptop** class with **brand**, **model**, **ram_gb** and **price**. Create three separate objects and change the price of one object.
3. Create two objects with the same attribute values. Use **is** to check whether they are the same object.
4. Add a default value to at least one **__init__** parameter.
5. Create one object using keyword arguments.

Part B - Methods and state

1. Extend your **Book** class with an **is_long()** method that returns True if the book has more than 300 pages.
2. Create a **BankAccount** class with **owner** and **balance**. Add a **deposit()** method that changes the balance.
3. Add a **withdraw()** method. Prevent withdrawals that would make the balance negative by raising a **ValueError**.
4. Create a **Task** class with **title** and **completed=False**. Add **complete()** and **reopen()** methods.
5. Create at least two objects from one of your classes and show that changing the state of one object does not change the other.

Part C - Instance and class attributes

1. Create a **Product** class with **name** and **price** as instance attributes.
2. Add a class attribute called **tax_rate** that is shared by all Product objects.
3. Add a **price_with_tax()** method that returns the price including tax.
4. Create at least three Product objects and print their prices with tax.
5. Change **Product.tax_rate** and show how it affects the Product objects.
6. Give one Product object its own **tax_rate**. Print the tax rate from that object, another Product object and the Product class.

Part D - Collections of objects

1. Create at least six **Student** objects with **name** and **score**.
2. Store all Student objects in a list.
3. Loop through the list and print each student's name and score.
4. Add a **get_status()** method that returns "PASS" or "FAIL" based on the score.
5. Loop through the students again and print each student's name and status.
6. Use a list comprehension to create a new list containing only students with a score of 70 or higher.

Part E - Objects inside objects

1. Create a **Teacher** class with a name.
2. Create a **Course** class with a course name and a teacher. The teacher should be a Teacher object.
3. Create a Teacher object and use it when creating a Course object.
4. Print the course name and the teacher's name through the Course object.
5. Extend Course so that it also contains an initially empty list of Student objects.
6. Add an **add_student()** method and use it to add at least three Student objects to the course.
7. Loop through **course.students** and print the name of every student.

Part F - Applied challenge: Course manager

1. Build a small course management program using **Student**, **Teacher** and **Course** classes.
2. Student should contain at least **name** and **score**.
3. Student should have a method that returns "PASS" or "FAIL".
4. Teacher should contain at least a **name**.
5. Course should contain a **name**, a Teacher object and a list of Student objects.
6. Add methods for adding a student and showing how many students are currently in the course.
7. Add a method that returns a list containing only the students who passed.
8. Add validation somewhere in your program using **ValueError**. Choose a validation that makes sense.
9. Create at least five Student objects, one Teacher object and one Course object. Demonstrate that your methods work.
10. Print a simple course summary containing the course name, teacher name, number of students and the names of the students who passed.

Part G - Stretch challenges

1. Add a method that updates a student's score with validation.
2. Add a method to Course that finds students above a score threshold.
3. Create another Course object and show that its student list is separate from the first course.
4. Add one useful class attribute to Student, Teacher or Course and explain in a comment why it belongs to the class rather than an individual object.